



Unidad 1

Desarrollo de software

Módulo: Entorns de Desenvolupament (EDE)

Ciclo formativo: 1r DAW (Desarrollo de Aplicaciones Web)

Curso: 2026-2027

Docente: Noel Marco Biendicho

Centro: IES Sant Vicent Ferrer — Algemesí

Índice

1. El programa y los componentes del sistema (CA 1a)
2. El ciclo de vida del software: fases del desarrollo (CA 1b)
3. Código fuente, código objeto y código ejecutable (CA 1c)
4. Código intermedio y máquinas virtuales (CA 1d)
5. Clasificación de los lenguajes de programación (CA 1e)
6. Herramientas del desarrollo de software (CA 1f)
7. Reto de clase
8. Resumen de la unidad
9. Para saber más

RA cubierto: RA1 — *Reconoce los elementos y herramientas que intervienen en el desarrollo de un programa informático, analizando sus características y las fases en que actúan hasta llegar a su puesta en funcionamiento* (CA 1a-1f)

 **Nota de programación:** el CA 1g (metodologías ágiles de desarrollo de software) **no** se trabaja en esta unidad — está asignado a la UP6 (Gestión de proyectos informáticos), donde el alumnado ya conoce el ciclo de vida y las fases que se presentan aquí.

 **Nota de estudio:** esta unidad da el vocabulario y los conceptos que usarás durante todo el curso — cuando en UD2 instales un IDE o en UD3 hagas tu primer commit, estarás usando "herramientas de desarrollo" tal como se clasifican aquí. No hay apenas código; el objetivo es que sepas *de qué estamos hablando* antes de ponernos a programar.

1. El programa y los componentes del sistema (CA 1a)

1.1. Informática, software y programa

La **informática** es el tratamiento automático de la información mediante un dispositivo. Toda esa información se representa internamente en **binario** (secuencias de 0 y 1): la unidad mínima es el **bit**, y 8 bits forman un **byte**.

Estas unidades reaparecen todo el curso — el tamaño de un archivo, la capacidad de una base de datos, cuánto ocupa una imagen que subes a un dashboard — así que conviene tenerlas claras desde ya:

Unidad	Equivale a
1 byte	8 bits
1 KB (kilobyte)	1024 bytes
1 MB (megabyte)	1024 KB
1 GB (gigabyte)	1024 MB

 **Ejercicio 1 — Unidades de información con un caso real** Un dashboard de finanzas personales guarda cada transacción como una fila de unos 200 bytes en una base de datos. Calcula, mostrando las operaciones: (a) cuántos KB ocupará la base de datos de un usuario que registra 15 transacciones al día durante un año; (b) cuántos MB ocuparía si la aplicación tuviera 10.000 usuarios haciendo lo mismo. ¿Te sorprende el resultado?

El **software** es la parte intangible de un sistema informático: el conjunto de instrucciones que le dicen al hardware qué hacer. Dentro del software conviene distinguir varios niveles:

Término	Qué es	Ejemplo
Programa	Secuencia de instrucciones con una finalidad concreta, que devuelve un valor o realiza una función	Una función que suma dos números
Librería	Archivo que empaqueta programas ya escritos, reutilizables desde otro código	Un archivo <code>.dll</code> , un paquete <code>pip</code>

Término	Qué es	Ejemplo
Aplicación	Uno o varios programas (con o sin librerías) orientados a una tarea del usuario final	Adobe Photoshop
Suite	Varias aplicaciones independientes distribuidas juntas	LibreOffice, MS Office

⚠ **Importante:** el sistema operativo no encaja en una sola de estas categorías — es, en la práctica, una suite de programas, librerías y aplicaciones que gestionan el resto del sistema.

🧩 **Ejercicio 2 — Vocabulario técnico correcto** Corrige esta frase para que sea técnicamente precisa (identifica los 4 términos mal usados y explica por qué):

"En mi ordenador tengo programas como LibreOffice Writer o Adobe Photoshop, que en esencia envían órdenes al procesador. El primero forma parte de una librería llamada LibreOffice, y el segundo usa internamente varias suites donde tiene guardadas funciones de diseño gráfico. Ambos han sido codificados por programadores expertos."

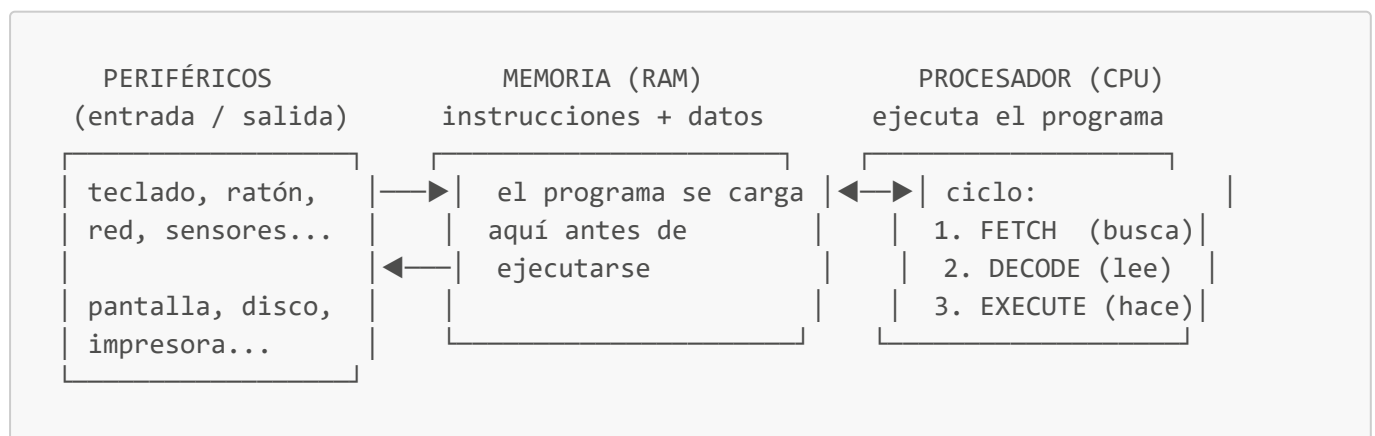
Otro eje de clasificación: software de sistema vs. software de aplicación. La tabla anterior distingue *qué forma tiene* el software (programa/librería/aplicación/suite); esta otra distingue *para qué sirve*, y es igual de necesaria — un controlador, por ejemplo, es una librería (por su forma) que pertenece al software de sistema (por su función):

Tipo	Función	Ejemplos
Software de sistema	Gestiona el hardware y da la base sobre la que corre todo lo demás	Sistema operativo, controladores (<i>drivers</i>), firmware (BIOS/UEFI), utilidades del sistema
Software de aplicación	Resuelve tareas concretas del usuario final, apoyándose en el software de sistema	Navegador, editor de código, un dashboard que tú programes

🧩 **Ejercicio 3 — Software de sistema o de aplicación** Abre el listado de programas instalados de tu ordenador (o el gestor de aplicaciones del móvil) y elige 6: 3 que consideres software de sistema y 3 que consideres software de aplicación. Para cada uno, justifica en una frase por qué lo has clasificado así, y di además si por su forma es un programa, una librería, una aplicación o una suite.

1.2. Cómo un programa usa el hardware: memoria, procesador y periféricos

Cuando un programa se ejecuta, no lo hace en el vacío: necesita apoyarse en tres componentes del sistema.



- **Memoria (RAM):** mientras un programa se ejecuta, tanto sus instrucciones como los datos que maneja viven en memoria principal. Al cerrarlo, ese contenido se pierde (es memoria *volátil*) — por eso los datos que queremos conservar hay que guardarlos en un periférico de almacenamiento (disco).
- **Procesador (CPU):** ejecuta el programa repitiendo el llamado **ciclo de instrucción**: *fetch* (busca la siguiente instrucción en memoria), *decode* (la interpreta) y *execute* (la realiza, usando sus registros internos y la unidad aritmético-lógica).
- **Periféricos:** son la vía de entrada y salida del programa con el exterior. De **entrada** (teclado, ratón, red, sensores) el programa recibe datos; de **salida** (pantalla, impresora, altavoces) o de **almacenamiento** (disco, red) el programa devuelve o persiste resultados.

No toda la memoria es RAM. La RAM es la que más se usa mientras el programa corre, pero conviene distinguirla de otros dos tipos:

Tipo	¿Volátil?	Qué guarda	Ejemplos
RAM	Sí (se pierde al apagar)	Instrucciones y datos del programa en ejecución	Memoria del ordenador
ROM / firmware	No	Instrucciones básicas de arranque, grabadas de fábrica	BIOS/UEFI
Almacenamiento secundario	No	Datos y programas de forma permanente	Disco duro, SSD, almacenamiento en red

 **Vigente:** la mayoría de procesadores actuales incorporan además una memoria **caché**, mucho más rápida que la RAM pero muy pequeña, que guarda los datos e instrucciones de uso más frecuente para evitar viajes constantes a la RAM. Es una de las razones por las que dos CPU con la misma velocidad de reloj pueden rendir de forma muy distinta.

Los periféricos tampoco son solo "entrada" o "salida". Algunos dispositivos cumplen las dos funciones a la vez:

Tipo	Función	Ejemplos
Entrada	El programa recibe datos del exterior	Teclado, ratón, micrófono
Salida	El programa envía resultados al exterior	Pantalla, altavoces, impresora
Entrada/salida	Cumple ambas funciones	Pantalla táctil, tarjeta de red, disco externo

 **Actividad rápida (5 min):** abre el Administrador de tareas (Windows) o <http://top> (Linux) y busca: ¿cuánta RAM está usando el navegador ahora mismo? ¿Qué proceso usa más CPU? Relaciona lo que ves con el ciclo fetch-decode-execute y con la volatilidad de la RAM — si fuerzas el cierre de un proceso sin guardar, ¿qué se pierde y por qué?

 **Por qué importa hoy:** cuando más adelante programemos, por ejemplo, un dashboard de finanzas personales, esta relación se ve literalmente: los datos que el usuario teclea entran por un periférico de entrada, se guardan temporalmente en variables (memoria RAM) mientras el programa calcula, y el resultado final se persiste en una base de datos en disco (almacenamiento secundario) — sin este esquema, "guardar" y "mostrar en pantalla" son solo palabras sueltas.

 **Ejercicio 4 — Memoria, procesador y periféricos en un caso real** Piensa en una aplicación sencilla que pida un nombre por teclado y muestre en pantalla un saludo. Identifica, para ese programa: qué periférico interviene en la entrada, qué periférico interviene en la salida, y en qué momento el dato "nombre" pasa por la memoria RAM y en qué momento el procesador ejecuta una instrucción.

1.3. Programar vs. desarrollar software

Programar es, en esencia, codificar instrucciones para que un dispositivo se comporte de una manera concreta. **Desarrollar software** es bastante más: incluye analizar, diseñar, probar, documentar y mantener, además de codificar. Por eso en este módulo hablaremos de **desarrollador** en vez de "programador" — la codificación es solo una de las fases (lo vemos en el punto 2).

De la idea a la solución. Incluso antes de llegar a las fases formales del punto 2, resolver cualquier problema de programación — por pequeño que sea — sigue en miniatura el mismo esqueleto:

1. **Entender el problema:** ¿qué se pide exactamente? (a menudo no está tan claro como parece — lo veremos en el punto 2, fase de Análisis)
2. **Diseñar una solución:** pensar el "cómo" en términos generales, sin escribir código todavía (a veces con pseudocódigo o un diagrama)
3. **Implementarla:** traducir esa solución a un lenguaje de programación concreto — esto es, estrictamente, "programar"
4. **Comprobarla:** verificar que funciona y que hace lo que debía

 Este proceso de 4 pasos es el mismo esqueleto, en miniatura, que el ciclo de vida del software completo que viene a continuación — análisis, diseño, codificación y pruebas no son ideas exclusivas de proyectos grandes: aparecen incluso al resolver un ejercicio pequeño.

2. El ciclo de vida del software: fases del desarrollo (CA 1b)

Todo desarrollo de software recorre, con más o menos rigor, un conjunto de etapas conocido como **ciclo de vida del software**. El modelo más clásico —y el que usaremos como referencia para entender cada fase— es el **modelo en cascada**: cada fase se apoya en la anterior y genera una documentación propia, sin empezar la siguiente hasta cerrar la actual.

2.1. Análisis

Se recogen y documentan los requisitos del cliente. Es una comunicación **bilateral**: el cliente rara vez sabe expresar con precisión técnica lo que necesita, así que el analista tiene que indagar, no solo tomar nota. Las técnicas más habituales son la **entrevista**, el **cuestionario**, la **observación** del proceso actual y la revisión de documentación ya existente.

Los requisitos recogidos se dividen en dos tipos, y confundirlos es uno de los errores más comunes al empezar:

Tipo	Qué describe	Ejemplo (dashboard de finanzas personales)
Funcional	Qué debe <i>hacer</i> el sistema — una acción o función concreta	"El usuario puede registrar un gasto indicando importe, fecha y categoría"

Tipo	Qué describe	Ejemplo (dashboard de finanzas personales)
No funcional	Cómo debe <i>comportarse</i> el sistema al hacerlo — rendimiento, seguridad, usabilidad, disponibilidad...	"El listado de gastos debe cargar en menos de 2 segundos"; "los datos se cifran en tránsito"

Todo esto se recoge en la **ERS (Especificación de Requisitos del Sistema)**, junto a un primer boceto de las entidades del sistema (diagrama E/R o de clases preliminar).

 **Qué sigue vigente y qué es ya historia:** durante décadas, el estándar de referencia para redactar una ERS fue **IEEE 830** (1998) — hoy retirado. Lo sustituye **ISO/IEC/IEEE 29148**, la norma vigente para especificar requisitos de software. El contenido de fondo (qué es un requisito funcional o no funcional) apenas ha cambiado; lo que cambia es el estándar formal que regula cómo documentarlo.

2.2. Diseño

Se define el funcionamiento del sistema **sin entrar todavía en el código**, a dos niveles:

- **Diseño arquitectónico:** la visión global — en qué módulos o capas se organiza el sistema y qué tecnología usa cada uno (por ejemplo, separar un dashboard de finanzas en una capa de interfaz, una de lógica de negocio y una de acceso a la base de datos).
- **Diseño detallado:** dentro de cada módulo, cómo se resuelve internamente — qué estructuras de datos y qué algoritmos usa cada función, todavía en diagramas o pseudocódigo, no en código real.

La notación estándar vigente para representar estos diseños es **UML (Unified Modeling Language)**, con distintos diagramas según lo que se quiera mostrar: **de casos de uso** (qué puede hacer cada tipo de usuario), **de secuencia** (en qué orden se comunican los componentes) y **de clases** (qué entidades existen y cómo se relacionan). Este último es también donde se formaliza el diseño de la base de datos — para el dashboard de finanzas, por ejemplo, las entidades *Usuario*, *Transacción* y *Categoría*, y cómo se relacionan entre sí.

Todo este trabajo de diseño queda recogido en el **cuaderno de carga**: el documento técnico, consensuado con el cliente, que sirve de base para empezar a programar.

2.3. Codificación

Se traduce el cuaderno de carga al lenguaje de programación elegido. "Código fuente comentado" no significa solo añadir comentarios: implica seguir unas mínimas buenas prácticas para que cualquier otra persona (o tú mismo, meses después) pueda entenderlo:

- **Nombres significativos:** `calcularTotalGastos()` en vez de `f1()`.
- **Comentarios que expliquen el porqué**, no el qué — el código ya dice *qué* hace; el comentario debe aportar el motivo cuando no sea obvio.
- **Convenciones de estilo** consistentes, normalmente las que marca el propio lenguaje o el equipo (por ejemplo, PEP 8 en Python).

 **Por qué importa hoy:** es en esta fase donde entra de forma natural el **control de versiones** (Git) — cada avance de código se registra como un cambio con su propio historial. Lo veremos en detalle en la UP3, pero el hábito de comentar y nombrar bien empieza aquí, antes de tocar Git.

2.4. Pruebas

Se comprueba que el software no tiene errores y que hace lo que debía — idealmente lo prueba alguien distinto de quien programó, porque a quien ha escrito el código le cuesta más ver sus propios fallos (conoce el camino que "debería" funcionar y tiende a probar solo ese).

Según cuánto se conoce del código:

Tipo	Cómo se prueba	Se fija en...
Caja negra	Sin mirar el código interno — solo se dan entradas y se comprueba si la salida es la esperada	La funcionalidad, desde fuera
Caja blanca	Conociendo la estructura interna del código	Que se ejecuten todas las líneas y condiciones posibles (cobertura)

Según qué parte del sistema se prueba — de menor a mayor alcance:

Nivel	Qué se prueba
Pruebas unitarias	Una función o módulo por separado, de forma aislada
Pruebas de integración	Varios módulos trabajando juntos
Pruebas de sistema	La aplicación completa, de principio a fin
Pruebas de aceptación	El propio cliente comprueba que se cumple lo pactado en el Análisis

2.5. Documentación

Aquí se elabora la **documentación de usuario** — distinta de la documentación **técnica** que ya se ha ido generando en fases anteriores (ERS, cuaderno de carga, código comentado): manual de instalación, manual de uso, preguntas frecuentes...

 **Cambiando ahora mismo:** el manual de usuario extenso en PDF está perdiendo peso frente a la ayuda contextual integrada en la propia aplicación — tutoriales interactivos la primera vez que se abre, tooltips junto a cada opción, un asistente que guía paso a paso. La documentación no ha desaparecido: se ha movido dentro del propio producto.

2.6. Explotación

El software se instala en el entorno real de uso. Si sustituye a una versión anterior, existen varias estrategias de implantación, cada una con un compromiso distinto entre riesgo y velocidad:

Estrategia	En qué consiste
Directa	Se sustituye la versión anterior de golpe, para todos los usuarios a la vez
En paralelo	Ambas versiones conviven durante un tiempo, para comparar y dar seguridad
Piloto	Se implanta primero en un grupo reducido de usuarios antes de generalizar
Por fases	Se despliega módulo a módulo, no todo el sistema de una vez



Cambiando activamente: en desarrollo web, este concepto se traduce hoy en el **despliegue** (*deployment*) del código a un servidor, cada vez más automatizado mediante prácticas de **CI/CD** (integración y despliegue continuos) — lo veremos con detalle en unidades posteriores, pero la idea de fondo (llevar el software del entorno de desarrollo al real, con algún grado de gradualidad o seguridad) es la misma que en explotación.

2.7. Mantenimiento

Se actúa sobre el software ya en producción. Se distinguen cuatro tipos, y no todos significan "hay un error":

Tipo	Qué resuelve	Ejemplo
Correctivo	Corregir errores detectados en producción	"Se corrige un fallo al iniciar sesión"
Evolutivo	Añadir funcionalidad nueva que el cliente no pidió al principio	"Se añade un modo oscuro"
Adaptativo	Adaptarse a cambios del entorno, no del propio software	"Se adapta a la nueva versión de Android"
Perfectivo	Mejorar sin cambiar la funcionalidad visible	"Se optimiza el tiempo de arranque"

Si el mantenimiento implica una ampliación importante, suele ser necesario revisar fases anteriores — Análisis y Diseño incluidos.



Fíjate en el registro de actualizaciones de cualquier app que uses: casi cada entrada encaja en uno de estos cuatro tipos. Es justo lo que te va a pedir el reto de clase de esta unidad.

¿Quién interviene en cada fase? Los roles no son compartimentos estancos — la misma persona puede cubrir varios:

Rol	Fases en las que participa
Analista de sistemas	Análisis
Diseñador de software	Diseño
Analista programador ("desarrollador")	Diseño y Codificación
Programador	Codificación
Arquitecto de software	Análisis, Diseño, Documentación y Explotación

¿Cascada es la única forma de organizarlo?

No. El modelo en cascada es lineal y rígido — si en Pruebas aparece un fallo de Diseño, hay que volver atrás varias fases. Existen alternativas clásicas que organizan estas mismas fases de otra manera:

Modelo	Idea central
Iterativo-incremental	El software se construye y entrega en incrementos sucesivos, cada uno añadiendo funcionalidad sobre el anterior
En espiral (Boehm)	Combina iteración con un análisis de riesgos explícito en cada vuelta, antes de seguir avanzando
De prototipado	Se construye pronto una versión reducida y funcional para validarla con el cliente antes de desarrollar el sistema completo

 **Actualidad:** hoy, la gran mayoría de equipos de desarrollo no usa cascada ni estos modelos clásicos como forma habitual de trabajar, sino **metodologías ágiles** (Scrum, Kanban...) — según el informe *State of Agile* de Digital.ai, alrededor del 71% de las organizaciones declara usarlas en su ciclo de desarrollo. La cascada sigue siendo útil como modelo de referencia para entender qué fases existen y qué produce cada una (que es el objetivo de este punto), y todavía se usa en proyectos con requisitos muy cerrados o muy regulados — pero no es representativa de cómo trabaja hoy la mayoría de equipos. Cómo funcionan las metodologías ágiles en concreto se ve en detalle en la UP6 (CA 1g).

Ejercicios de la fase

Ejercicio 5 — Fases del ciclo de vida aplicadas Un cliente te pide una aplicación de gestión de gastos personales. Redacta brevemente (2-3 líneas por fase) qué harías en cada una de las 7 fases del ciclo de vida para ese proyecto concreto.

Ejercicio 6 — Requisitos funcionales o no funcionales Para la misma aplicación de gastos personales, clasifica estos 6 requisitos en funcionales o no funcionales, justificando cada uno en una frase: (1) "el usuario puede exportar sus gastos a PDF"; (2) "la aplicación debe funcionar sin conexión a internet"; (3) "se pueden crear categorías personalizadas"; (4) "ningún usuario puede ver los datos de otro"; (5) "el histórico de un año debe cargar en menos de 3 segundos"; (6) "se puede iniciar sesión con Google".

Ejercicio 7 — Clasifica el mantenimiento Busca el registro de cambios (*changelog* o "novedades de esta versión") de una app que tengas instalada y elige 4 entradas distintas. Clasifica cada una como mantenimiento correctivo, evolutivo, adaptativo o perfectivo, y justifícalo.

3. Código fuente, código objeto y código ejecutable (CA 1c)

Un dispositivo solo entiende **código máquina** (binario). Como nadie programa directamente en binario, escribimos en un lenguaje de programación y dejamos que unas herramientas lo traduzcan:

```
código fuente  —(compilador)—▶ código objeto  —(+ librerías, enlazador)—▶
código ejecutable
  (.java)                (.class)
(específico del SO)
```

- **Código fuente:** el conjunto de instrucciones que escribe el desarrollador, en un fichero de texto (`.java`, `.py`, `.c...`).

- **Código objeto** (o *código intermedio*, o *bytecode*): resultado de compilar el código fuente. Todavía no es directamente ejecutable por el hardware — en Java, por ejemplo, es el fichero `.class`.
- **Código ejecutable**: se obtiene al añadir al código objeto las funciones de librerías usadas y las particularidades del sistema operativo de destino (proceso normalmente realizado por un *enlazador*). Es lo que el dispositivo interpreta directamente.

⚠ Un mismo código fuente puede necesitar generar **ejecutables distintos** según el sistema operativo destino (Windows, Linux, macOS), aunque el código fuente no cambie.

El enlazador hace algo más que "juntar" ficheros. Decide además *cómo* se incorporan las librerías al ejecutable final:

- **Enlazado estático**: el código de la librería se copia dentro del propio ejecutable. El fichero final es más grande, pero no depende de nada externo para funcionar.
- **Enlazado dinámico**: el ejecutable solo guarda una referencia a la librería, que se carga por separado en tiempo de ejecución (los `.dll` en Windows, los `.so` en Linux). Varios programas pueden compartir una misma copia de la librería en memoria, y actualizarla no obliga a recompilar cada programa que la usa.

El esquema no se ve igual en todos los lenguajes. Fuente → objeto → ejecutable es más visible en un lenguaje compilado; en uno interpretado, el propio término "ejecutable" no aplica de la misma forma:

Lenguaje	Código fuente	Código objeto / intermedio	Código ejecutable
C (compilado)	<code>programa.c</code>	Interno al proceso de compilación, no se guarda como archivo aparte	<code>programa.exe</code> / binario nativo
Java (híbrido)	<code>Programa.java</code>	<code>Programa.class</code> (bytecode)	No hay un ejecutable nativo — lo interpreta la JVM (punto 4)
Python (interpretado)	<code>programa.py</code>	<code>__pycache__/programa.cpython-3XX.pyc</code> (bytecode cacheado, automático)	No se genera — el intérprete lee y ejecuta línea a línea

💡 Retomaremos esta tabla en el punto 5, al clasificar los lenguajes según cómo se ejecutan — es la misma idea vista desde el otro lado.

En la práctica, con comandos reales:

```
# C: compilación + enlazado en un solo paso
$ gcc programa.c -o programa
$ ./programa

# Java: compilación a bytecode, y ejecución sobre la JVM (punto 4)
$ javac HolaMundo.java      # genera HolaMundo.class (código objeto)
$ java HolaMundo             # la JVM interpreta el bytecode

# Python: sin compilación explícita
$ python programa.py         # el intérprete lee y ejecuta línea a línea
```



Cambiando activamente: la frontera se difumina. Motores como V8 (el que usan Chrome y Node.js) ya no solo interpretan JavaScript línea a línea: compilan sobre la marcha, mientras el programa se ejecuta, las partes de código que más se repiten — se llama compilación **JIT** (*just-in-time*). En sentido contrario, herramientas como GraalVM permiten compilar bytecode Java directamente a un ejecutable nativo (**native-image**), sin necesitar una JVM en el equipo final. La clasificación compilado/interpretado/híbrido del punto 5 sigue siendo útil para entender la idea de fondo, pero cada vez hay más lenguajes que mezclan las tres estrategias.

 **Ejercicio 8 — De código fuente a ejecutable** Instala un JDK, escribe un programa `HolaMundo.java` que imprima tu nombre, y compílalo con `javac HolaMundo.java`. Localiza el fichero `.class` generado y ejecútalo con `java HolaMundo`. Explica, con tus palabras, qué representa cada uno de los tres ficheros/momentos (fuente, objeto, ejecución) en este proceso.

 **Ejercicio 9 — El esquema en distintos lenguajes** Apoyándote en la tabla anterior, explica con tus palabras por qué en Python no tiene sentido hablar de "código ejecutable" de la misma forma que en C, y qué papel juega el fichero `.pyc` que genera Python automáticamente la primera vez que ejecutas un script.

 **Ejercicio 10 — Compilado vs. interpretado, cronómetro en mano** Escribe un programa que sume los números del 1 al 10 millones, en C y en Python. Compila y ejecuta la versión en C (`gcc + ./programa`); ejecuta la versión en Python directamente. Cronometra ambas (`time ./programa` y `time python programa.py` en Linux/Mac; `Measure-Command` en PowerShell). ¿Qué diferencia observas y por qué, según lo visto en este punto?

4. Código intermedio y máquinas virtuales (CA 1d)

Según cómo se trate el código, un ejecutable puede ser:

- **Portable:** funciona en varias plataformas sin recompilar (p. ej., un `.jar` de Java, porque usa *bytecode* no ligado a un procesador concreto).
- **No portable:** pensado para una plataforma concreta (p. ej., un ejecutable compilado en C para Windows).

Algunos lenguajes, como Java, resuelven la portabilidad con una **máquina virtual**: una aplicación que emula un sistema (o parte de él) dentro del sistema operativo real.

Tipo de máquina virtual	Qué hace	Ejemplos
De sistema	Simula un ordenador completo (arquitectura + SO) dentro de otro	VirtualBox, VMware
De proceso	Ejecuta el código intermedio (bytecode) de un lenguaje concreto, independizándolo del hardware real	JVM (Java Virtual Machine)

La **JVM** interpreta y ejecuta el *bytecode* que genera el compilador de Java (los ficheros `.class`), traduciéndolo sobre la marcha a las instrucciones del hardware concreto donde se ejecuta. Por eso el mismo `.class` corre en Windows, Linux o macOS sin recompilar, siempre que haya una JVM instalada: "*write once, run anywhere*".

 **Por qué existe la JVM:** a mediados de los años 90 cada sistema operativo tenía su propio formato de ejecutable, y distribuir un programa para varias plataformas obligaba a compilarlo y mantenerlo por separado para cada una. Sun Microsystems diseñó Java (1995) con la JVM como pieza central para resolver justo ese problema: compilar una sola vez a bytecode y dejar que la máquina virtual se encargue de adaptarse al sistema real. La necesidad que resolvió sigue vigente — es la misma razón de fondo por la que hoy existen los contenedores y WebAssembly, que vemos a continuación.

El pipeline completo, de principio a fin:

```
Programa.java —(javac, compilador)—► Programa.class (bytecode)
Programa.class —(java, se ejecuta sobre la JVM)—► el programa corre
```

 **Sigue vigente:** el modelo de máquina virtual de proceso no es exclusivo de Java — Python (con su intérprete y bytecode `.pyc`) y .NET (CLR) funcionan sobre una idea muy similar.

 **Cambiando activamente: contenedores.** Una máquina virtual de sistema emula un ordenador completo con su propio sistema operativo — aísla mucho, pero es "pesada". Una alternativa moderna y más ligera son los **contenedores** (Docker es el más conocido): en vez de emular todo el hardware, comparten el núcleo del sistema operativo anfitrión y solo aíslan la aplicación y sus dependencias. Los veremos con detalle en unidades de despliegue más adelante — de momento basta con saber que "aislar un programa del resto del sistema" no siempre pasa por una máquina virtual completa.

 **Cambiando activamente: WebAssembly (Wasm).** Es la evolución más reciente de la misma idea de "bytecode portable", pero pensada para el navegador: un formato que permite ejecutar código escrito en C, C++, Rust o incluso Java dentro de una página web, a una velocidad cercana a la nativa — algo que JavaScript por sí solo no ofrece. No sustituye a JavaScript, lo complementa en las partes de una aplicación web que necesitan más rendimiento (edición de imagen o vídeo en el propio navegador, videojuegos, cálculo intensivo). En el desarrollo web actual es donde más está creciendo esta idea de "máquina virtual de proceso".

 **Ejercicio 11 — JVM, WebAssembly y portabilidad** Compara la JVM con WebAssembly: busca una definición breve de Wasm y explica, con tus propias palabras, en qué se parecen (ambas son máquinas virtuales de proceso que ejecutan bytecode portable) y en qué se diferencian (para qué se usa cada una y dónde se ejecuta). ¿Por qué crees que un desarrollador de aplicaciones web actual debería conocer al menos la existencia de WebAssembly?

5. Clasificación de los lenguajes de programación (CA 1e)

No hay una única forma de clasificar los lenguajes; estas son las más habituales.

Según cómo se ejecutan:

Tipo	Cómo funciona	Ejemplos	Dónde se usan más
Compilados	El código fuente se traduce entero, de una vez, a un ejecutable	C, C++, C#, Pascal	Software de escritorio (ejecución rápida, ficheros más pesados)

Tipo	Cómo funciona	Ejemplos	Dónde se usan más
Interpretados	Un intérprete traduce y ejecuta el código línea a línea, sin generar ejecutable propio	Python, PHP, JavaScript	Entornos web y scripting (menos recursos, más lentos)
Híbridos / virtuales	Se compilan a código intermedio (bytecode), que luego interpreta una máquina virtual	Java, C# (en parte)	Aplicaciones que deben correr en varias plataformas sin recompilar

Según el nivel de abstracción (cuánto se parecen al lenguaje natural frente al lenguaje máquina):

Nivel	Característica	Ejemplos
Bajo nivel	Instrucciones muy cercanas al hardware, acceso directo a registros	Ensamblador
Medio nivel	Permite acceder a memoria y registros, pero con una sintaxis más legible	C
Alto nivel	Sintaxis cercana al lenguaje humano, abstrae los detalles del hardware	Python, Java, PHP

Según el paradigma de programación (la forma de plantear la solución):

Paradigma	Idea central	Ejemplo de lenguaje
Imperativo / procedimental	Se describe <i>paso a paso</i> cómo hacer algo	C
Orientado a objetos	Se modela el problema como objetos que interactúan	Java
Funcional	Se programa componiendo funciones, evitando el estado mutable	Haskell (y, parcialmente, JavaScript o Python)
Declarativo	Se describe <i>qué</i> se quiere obtener, no <i>cómo</i> obtenerlo	SQL

Un mismo problema, dos paradigmas. La diferencia entre imperativo y declarativo se nota mejor en código que en definiciones — para "quedarse con los números pares de una lista":

```
# Imperativo: se describe paso a paso el "cómo"
pares = []
for n in numeros:
    if n % 2 == 0:
        pares.append(n)
```

```
-- Declarativo: se describe el "qué", sin decir cómo recorrerlo
SELECT * FROM numeros WHERE n % 2 = 0;
```

En la práctica, pocos lenguajes son "puros". La tabla de paradigmas clasifica según la idea *dominante* de cada lenguaje, no una etiqueta única y excluyente: Python y JavaScript son fundamentalmente imperativos y orientados a objetos, pero incorporan funciones de estilo funcional (`map`, `filter`) sin ningún problema; SQL es declarativo, pero muchos motores permiten añadir lógica imperativa (procedimientos almacenados en PL/SQL, T-SQL...). Es habitual que un mismo lenguaje combine varios paradigmas según lo que convenga en cada parte del código.

Por último, en el contexto de aplicaciones web es habitual distinguir entre **front-end** (la parte visible, que corre en el navegador del usuario: HTML, CSS, JavaScript) y **back-end** (la lógica no visible, que corre en un servidor: Java, Python, PHP, SQL...). Quien domina cómodamente ambas partes se conoce como **desarrollador full-stack** — uno de los perfiles más demandados en desarrollo web hoy, y el que iremos trabajando progresivamente a lo largo de este ciclo.



Actualidad: la popularidad de los lenguajes cambia con el tiempo. El **índice TIOBE** (tiobe.com/tiobe-index) es una de las referencias más consultadas para ver qué lenguajes están en auge o en declive — pero es una fotografía mensual, no una verdad fija: conviene consultarlo actualizado en vez de memorizar un ranking.

 **Ejercicio 12 — Clasifica estos lenguajes** Para Python, Java, C y JavaScript, indica: tipo de ejecución (compilado/interpretado/híbrido), nivel de abstracción y paradigma principal. Justifica cada respuesta en una frase.

 **Ejercicio 13 — El mismo problema, dos paradigmas** Para el dashboard de finanzas personales, plantea la tarea "obtener los usuarios que han gastado más de 100 € este mes": escribe una solución imperativa en pseudocódigo (con un bucle y una condición) y una solución declarativa en SQL (una sola consulta `SELECT`). Compara ambas: ¿cuál describe el "cómo" y cuál el "qué"?

6. Herramientas del desarrollo de software (CA 1f)

Además del lenguaje, un desarrollador se apoya en herramientas que dan soporte a cada fase del ciclo de vida. Esta unidad solo las **clasifica**; profundizaremos en varias de ellas en unidades posteriores.

6.1. Editor de código / IDE

Un editor de texto simple (el Bloc de notas) también puede escribir código, pero un **IDE** (*Integrated Development Environment*) añade herramientas integradas para todo el ciclo de codificación: autocompletado, resaltado de sintaxis, detección de errores mientras escribes y depuración incorporada. Ejemplos: Visual Studio Code, Eclipse, IntelliJ. **Se trabaja en detalle en la UP2.**

6.2. Compilador / intérprete

Traduce código fuente a código objeto/ejecutable, o lo ejecuta línea a línea — es la herramienta que materializa todo lo visto en los puntos 3 y 5 de esta unidad. Ejemplos: `javac/java`, el propio intérprete de Python, GCC. **UP1-UP2.**

6.3. Control de versiones

Registra el historial de cambios del código y permite que varias personas trabajen sobre el mismo proyecto sin sobrescribirse — ya lo adelantamos en el punto 2.3, al hablar de la fase de Codificación. Ejemplos: Git,

GitHub. **UP3.**

```
$ git init
$ git add cambios.py
$ git commit -m "Primer commit"
```

6.4. Depuración y pruebas

Permite ejecutar el programa paso a paso, inspeccionar el valor de las variables en cada momento, y automatizar las comprobaciones vistas en el punto 2.4 (pruebas unitarias, de integración...) en vez de repetirlas a mano cada vez. Ejemplos: el depurador integrado del IDE, JUnit. **UP5.**

6.5. Gestión de dependencias y construcción del proyecto

Instala librerías externas (retomando el concepto de librería del punto 1.1) sin tener que descargarlas e integrarlas a mano, y automatiza pasos repetitivos como compilar, ejecutar pruebas o empaquetar el proyecto. Ejemplos: **pip**, Maven, npm. **UP2.**

```
$ pip install matplotlib          # instala una librería
$ pip freeze > requirements.txt  # registra las dependencias del proyecto
```

6.6. Análisis y documentación de código

Revisa automáticamente la calidad del código (código duplicado, mal estilo, posibles vulnerabilidades) y genera documentación técnica a partir de los propios comentarios del código fuente. Ejemplos: SonarQube, Javadoc. **UP7.**

 **Cambiando activamente: asistentes de código con IA.** Herramientas como GitHub Copilot, o el autocompletado inteligente ya integrado en muchos IDEs actuales, sugieren código mientras escribes apoyándose en modelos de lenguaje. No son una categoría nueva: se integran dentro del editor/IDE (6.1). Están cambiando cómo se escribe código día a día, pero siguen haciendo falta los mismos fundamentos de esta unidad para revisar y entender lo que sugieren — no para copiarlo sin más.

 Ejemplo integrador: si más adelante desarrollamos un **dashboard de finanzas personales** en Python con una base de datos SQLite, usaríamos como mínimo: un IDE (VS Code) para escribir el código, Git/GitHub para el control de versiones, **pip** para instalar librerías (por ejemplo, para generar gráficos) y **pytest** para probar que los cálculos son correctos. Cada una de esas herramientas cubre una funcionalidad distinta dentro del desarrollo.

 **Ejercicio 14 — Herramientas para un proyecto** Para el proyecto de "dashboard de finanzas personales" del punto 6, indica qué herramienta de cada categoría de la tabla usarías y por qué, aunque hoy no sepas usarlas todavía (búscalo si hace falta).

 **Ejercicio 15 — Instala y prueba una herramienta** Elige una categoría de la tabla anterior con una herramienta que no conozcas todavía (por ejemplo, un linter de análisis de código o un gestor de dependencias distinto al visto en clase) e instálala en tu equipo. Documenta en 4-5 líneas: qué problema

resuelve, qué comando usaste para instalarla, y el resultado de haberla ejecutado una vez sobre un fichero o proyecto de prueba.

Reto de clase

Elige una aplicación que uses habitualmente en el móvil (una app de mensajería, de banca, de transporte...) y elabora una ficha de una página que incluya:

1. Una hipótesis razonada sobre qué lenguaje(s) de programación se usaron para su parte visible y para su lógica interna (front-end/back-end), con al menos un argumento técnico de por qué.
2. Una hipótesis sobre si el ejecutable que corre en tu móvil es compilado, interpretado o híbrido (justifícalo con lo que sepas de Android/iOS y las máquinas virtuales vistas en el punto 4).
3. Un esquema de las 7 fases del ciclo de vida aplicado a esa app: qué requisitos crees que se recogieron en el Análisis, y qué tipo de Mantenimiento recibe (busca su historial de actualizaciones en la tienda de aplicaciones como pista).

Variación evaluable: se puede pedir por parejas, comparando dos apps de la misma categoría (por ejemplo, dos apps bancarias) para que el alumnado contraste hipótesis distintas.

Resumen de la unidad

1. El programa y los componentes del sistema El software se clasifica por su forma (programa, librería, aplicación, suite) y por su función (software de sistema vs. de aplicación). Un programa en ejecución se apoya en memoria (RAM, volátil), procesador (ciclo fetch-decode-execute) y periféricos (entrada, salida o ambas) — programar es solo la fase de codificación dentro de desarrollar software, que además analiza, diseña, prueba, documenta y mantiene.

2. El ciclo de vida del software Modelo en cascada, en 7 fases: análisis (requisitos funcionales/no funcionales), diseño (arquitectónico y detallado, UML), codificación (buenas prácticas), pruebas (caja negra/blanca, niveles), documentación de usuario, explotación (estrategias de implantación) y mantenimiento (correctivo, evolutivo, adaptativo, perfectivo) — cada fase genera su propia documentación y con roles que pueden solaparse. La cascada es el modelo de referencia para entender las fases, pero hoy la mayoría de equipos usa metodologías ágiles (CA 1g, se ven en la UP6); otros modelos clásicos (iterativo-incremental, en espiral, prototipado) sí forman parte de esta unidad.

3. Código fuente, objeto y ejecutable El código fuente se compila a código objeto (bytecode), y este se enlaza (de forma estática o dinámica) con librerías y particularidades del SO para dar el ejecutable final — un esquema que se ve distinto según el lenguaje sea compilado, híbrido o interpretado, y que puede necesitar ejecutables distintos según la plataforma destino. La frontera compilado/interpretado ya no es tan tajante: motores JIT como V8 compilan JavaScript sobre la marcha, y herramientas como GraalVM compilan bytecode Java a ejecutable nativo.

4. Código intermedio y máquinas virtuales Las máquinas virtuales (de sistema o de proceso) permiten portabilidad: la JVM ejecuta el mismo bytecode `.class` en cualquier sistema operativo, la idea detrás de "write once, run anywhere" — Python y .NET siguen un planteamiento similar, y nació para resolver la falta de portabilidad entre sistemas operativos de los años 90. Los contenedores (Docker) y WebAssembly (bytecode portable para el navegador) son evoluciones más recientes de la misma idea.

5. Clasificación de los lenguajes Se clasifican según cómo se ejecutan (compilados, interpretados, híbridos), su nivel de abstracción (bajo, medio, alto) y su paradigma dominante (imperativo, orientado a objetos, funcional, declarativo — la mayoría combina varios, y un mismo problema se resuelve de forma distinta en cada paradigma) — además de la distinción front-end/back-end, y el perfil full-stack que domina ambas.

6. Herramientas del desarrollo de software IDE, compiladores/intérpretes, control de versiones, depuración y pruebas, gestión de dependencias, y análisis/documentación de código — cada categoría da soporte a una fase distinta del ciclo de vida del punto 2, con comandos y ejemplos reales (Git, pip) que se trabajarán en detalle a partir de la UP2; los asistentes de código con IA son la novedad más activa dentro del editor/IDE, sin sustituir los fundamentos de esta unidad.



Para saber más (opcional, no evaluable)

- Índice TIOBE (popularidad de lenguajes, actualizado mensualmente): <https://www.tiobe.com/tiobe-index/>
- Documentación oficial de la JVM: <https://docs.oracle.com/javase/specs/>